

Lab 5 - Part 2: Model Training & Experiment Tracking

- **Course:** Engineering of Intelligent Models
- **Module:** M3. Model Orchestration & Automation
- **Focus:** LSTM/GRU, Prophet, Hydra, and MLflow Tracking
- **Branch:** Lab5

1. Goal of the Laboratory

In this second segment, we shift our focus to the core of the machine learning lifecycle: **Model Training**. Because weather data is inherently sequential (time-series), traditional machine learning algorithms often fall short. We will implement three distinct algorithmic approaches:

1. **LSTM (Long Short-Term Memory):** A Recurrent Neural Network (RNN) designed to learn long-term dependencies in sequences.
2. **GRU (Gated Recurrent Unit):** A streamlined, computationally efficient alternative to the LSTM.
3. **Prophet:** A decomposable time-series additive model developed by Facebook, specifically optimized for strong seasonal effects.

To maintain strict MLOps standards, we will avoid hardcoding any hyperparameters. We will utilize **Hydra** to construct interchangeable configuration profiles and use **MLflow** to automatically track our training loss curves and parameters.

Note: Model artifact saving and future forecasting will be addressed in Lab 6).

2. Hyperparameter Management with Hydra

We need a modular configuration structure. We will create a `model` configuration group in Hydra, allowing us to swap the entire algorithmic architecture with a single command-line argument (e.g., `model=lstm` vs `model=prophet`).

Step 1: Create a new folder `conf/model/` and add the following three YAML files.

First file: `conf/model/lstm.yaml` :

```
name: "LSTM"
input_size: 3 # Temperature, Humidity, Precipitation
hidden_size: 64
num_layers: 2
dropout: 0.2
learning_rate: 0.001
batch_size: 32
epochs: 1000 # Decrease if you want faster runs during testing, but ideally, this should be high for better convergence
sequence_length: 24 # Look back 24 hours to predict the next step
```

Second file: `conf/model/gru.yaml` :

```
name: "GRU"
input_size: 3
hidden_size: 64
num_layers: 2
dropout: 0.2
learning_rate: 0.001
batch_size: 32
epochs: 1000 # Decrease if you want faster runs during testing, but ideally, this should be high for better convergence
sequence_length: 24
```

Third file: `conf/model/prophet.yaml` :

```
name: "Prophet"
seasonality_mode: "multiplicative"
```

```

yearly_seasonality: True
weekly_seasonality: True
daily_seasonality: True
changepoint_prior_scale: 0.05
# Prophet does not use epochs or batch sizes, it fits on the whole DataFrame

```

Step 2: Update your main conf/config.yaml to set a default model.

Add this to your `defaults:` list at the end of the file:

```

defaults:
  # Other defaults...
  - model: lstm # Default to LSTM
  - _self_

```

3. LSTM and GRU Model Implementation with PyTorch Lightning

Now, to implement the LSTM and GRU models, we will use PyTorch Lightning for cleaner code and better training management. We will create separate model classes for LSTM and GRU.

Step 1: Do not forget to add PyTorch Lightning at the end of your `requirements.txt`:

```
lightning==2.6.1
```

and then run `pip install -r requirements.txt` to install it.

Step 2: Create `src/training/models/lstm.py`

```

import pytorch_lightning as pl
import torch
from torch import nn

class WeatherLSTM(pl.LightningModule):
    def __init__(self, cfg):
        super().__init__()
        # Saves the Hydra config to the Lightning module for easy access
        self.save_hyperparameters(logger=False)
        self.cfg = cfg

        # Core LSTM Architecture
        self.lstm = nn.LSTM(
            input_size=cfg.input_size,
            hidden_size=cfg.hidden_size,
            num_layers=cfg.num_layers,
            dropout=cfg.dropout if cfg.num_layers > 1 else 0.0,
            batch_first=True
        )
        # Regressor to map the hidden state back to the 3 target variables
        self.regressor = nn.Linear(cfg.hidden_size, cfg.input_size)
        self.criterion = nn.MSELoss()

    def forward(self, x):
        # x shape: (batch_size, sequence_length, input_size)
        lstm_out, _ = self.lstm(x)
        # We only care about the prediction at the final time step of the sequence
        predictions = self.regressor(lstm_out[:, -1, :])
        return predictions

    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x)
        loss = self.criterion(y_hat, y)

        # Log the Loss directly to MLflow via Lightning's Logger
        self.log("train_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        return loss

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.cfg.learning_rate)

```

Step 3: Create `src/training/models/gru.py`

(Duplicate the LSTM code, but change the class name to `WeatherGRU` and replace `nn.LSTM` with `nn.GRU`).

4. Prophet Wrapper

Unlike deep learning models that require 3D tensors, Prophet expects a simple Pandas DataFrame with exactly two columns: `ds` (timestamp) and `y` (target variable). We will create a wrapper class to standardize its interface.

Step 1: Same thing for Prophet: add `prophet` to your `requirements.txt` :

`prophet==1.3.0`

and then run `pip install -r requirements.txt` to install it.

Step 2: Create `src/training/models/fbprophet.py` :

```
from prophet import Prophet
import pandas as pd
import logging

logger = logging.getLogger(__name__)

class WeatherProphet:
    def __init__(self, cfg):
        self.cfg = cfg
        self.model = Prophet(
            seasonality_mode=cfg.seasonality_mode,
            yearly_seasonality=cfg.yearly_seasonality,
            weekly_seasonality=cfg.weekly_seasonality,
            daily_seasonality=cfg.daily_seasonality,
            changepoint_prior_scale=cfg.changepoint_prior_scale
        )

    def fit(self, df: pd.DataFrame, target_column: str):
        """Prepares the DataFrame and fits the Prophet model."""
        # Prophet strictly requires 'ds' and 'y' column names
        prophet_df = pd.DataFrame({
            'ds': pd.to_datetime(df['date']),
            'y': df[target_column]
        })

        logger.info(f"Fitting Prophet model for target: {target_column}...")
        self.model.fit(prophet_df)
        return self.model
```

5. The Training Pipeline & MLflow Integration

We now need a central script that acts as the traffic controller: reading the data, looking at the Hydra config, instantiating the correct model, and logging the experiment to MLflow.

Refactor old `src/training/train_model.py` file to this new version:

```
import hydra
from omegaconf import DictConfig, OmegaConf
import mlflow
import pandas as pd
import numpy as np
import torch
from torch.utils.data import TensorDataset, DataLoader
import pytorch_lightning as pl
from pytorch_lightning.loggers import MLFlowLogger
from datetime import datetime

from models.lstm import WeatherLSTM
from models.gru import WeatherGRU
from models.fbprophet import WeatherProphet
```

```

import logging
import warnings
import yaml

logger = logging.getLogger(__name__)

def create_sliding_windows(data, seq_length):
    """Transforms the multivariate time series data into sliding windows for sequence modeling."""
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        x = data[i:(i + seq_length)]
        y = data[i + seq_length]
        xs.append(x)
        ys.append(y)
    return torch.tensor(np.array(xs), dtype=torch.float32), torch.tensor(np.array(ys), dtype=torch.float32)

def get_dvc_hash(dvc_file_path="data/raw.dvc"):
    """Extracts the exact md5 data hash from the DVC tracking file."""
    try:
        with open(dvc_file_path, 'r') as f:
            dvc_data = yaml.safe_load(f)
            # Access the md5 hash of the tracked directory/file
            return dvc_data['outs'][0]['md5']
    except Exception as e:
        logger.warning(f"Could not read DVC hash from {dvc_file_path}: {e}")
        return "unknown_dvc_hash"

@hydra.main(version_base=None, config_path="../../conf", config_name="config")
def train(cfg: DictConfig):
    logger.info(f"--- Starting Training Pipeline for {cfg.model.name} ---")

    mlflow.set_tracking_uri("http://mlflow_server:5000")
    mlflow.set_experiment("Weather_Forecasting_Models")

    # Use Lisbon for now, for this Lab
    data_path = "data/raw/historical_weather-Lisbon.csv"
    df = pd.read_csv(data_path)

    # 2. Generate the dynamic run name with a timestamp
    current_time = datetime.now().strftime("%Y%m%d_%H%M%S")
    dynamic_run_name = f"{cfg.model.name}_Training_Run_{current_time}"

    # 3. Capture the active run context as 'run'
    with mlflow.start_run(run_name=dynamic_run_name) as run:
        # Log Hydra parameters in a clean, flattened format for MLflow
        mlflow.log_params(OmegaConf.to_container(cfg.model, resolve=True))
        # Log the DVC Data Hash for strict reproducibility
        data_hash = get_dvc_hash("data/raw.dvc")
        mlflow.log_param("data_dvc_hash", data_hash)

        if cfg.model.name in ["LSTM", "GRU"]:
            # Multivariate Forecasting
            features = df[['temperature_2m', 'relative_humidity_2m', 'precipitation']].values
            x, y = create_sliding_windows(features, cfg.model.sequence_length)
            dataset = TensorDataset(x, y)

            dataloader = DataLoader(
                dataset,
                batch_size=cfg.model.batch_size,
                shuffle=False,
                num_workers=4
            )

            model = WeatherLSTM(cfg.model) if cfg.model.name == "LSTM" else WeatherGRU(cfg.model)

```

```

# Pass the active run_id to the Lightning Logger
mlf_logger = MLFlowLogger(
    experiment_name="Weather_Forecasting_Models",
    tracking_uri="http://mlflow_server:5000",
    run_id=run.info.run_id # <--- This bridges them perfectly together!
)

trainer = pl.Trainer(
    max_epochs=cfg.model.epochs,
    logger=mlf_logger,
    enable_checkpointing=False,
    log_every_n_steps=5
)
trainer.fit(model, dataloader)

elif cfg.model.name == "Prophet":
    # Since Prophet is a univariate forecasting model, we'll focus only on temperature
    prophet_model = WeatherProphet(cfg.model)
    fitted_model = prophet_model.fit(df, target_column='temperature_2m')

    # 1. Generate predictions on the training data to calculate error
    prophet_df = pd.DataFrame({'ds': pd.to_datetime(df['date'])})
    forecast = fitted_model.predict(prophet_df)

    # 2. Calculate Mean Squared Error (MSE) using NumPy
    y_true = df['temperature_2m'].values
    y_pred = forecast['yhat'].values
    train_loss = np.mean((y_true - y_pred) ** 2)

    # 3. Explicitly log the metric to MLflow!
    mlflow.log_metric("train_loss", float(train_loss))
    logger.info(f"Prophet fitting complete. Train Loss (MSE): {train_loss:.4f}")

logger.info("--- Training Pipeline Complete ---")

if __name__ == "__main__":
    train()

```

Code Explanation:

- `create_sliding_windows()` : This helper function transforms our multivariate time-series data into the appropriate format for LSTM/GRU training. It creates sequences of past observations (of length `sequence_length`) as input features and the next time step as the target.
- `get_dvc_hash()` : This function reads the DVC tracking file (`raw.dvc`) to extract the exact md5 hash of the data being used.
- `data_path` : We are hardcoding the path to the Lisbon dataset for simplicity. In a production scenario, this could be parameterized or dynamically fetched from a data lake.
- `dynamic_run_name` : We create a unique run name for each training session by appending a timestamp. This makes it easy to identify runs in the MLflow UI.
- `mlflow.log_params(OmegaConf.to_container(cfg.model, resolve=True))` : This line logs all the hyperparameters from the Hydra config to MLflow in a structured way. This means that when you view the run in MLflow, you will see all the parameters (like `hidden_size`, `learning_rate`, etc.) neatly organized under the "Parameters" section.
- `mlflow.log_param("data_dvc_hash", data_hash)` : This logs the DVC data hash as a parameter in MLflow, ensuring that we have a clear record of which exact version of the data was used for this training run.
- `if cfg.model.name in ["LSTM", "GRU"]` : Depending on the model specified in the Hydra config, we instantiate either the LSTM or GRU model and train it using PyTorch Lightning. For Prophet, we fit the model only on the temperature data and calculate the training loss manually, since Prophet is strictly univariate and does not use epochs or batch sizes.
- `features = df[['temperature_2m', 'relative_humidity_2m', 'precipitation']].values` : We are using all three features for the LSTM and GRU models, which allows them to learn complex interactions between these variables. It looks back at the past 24 hours of all three features to predict the next hour's values for all three variables simultaneously.

- `x, y = create_sliding_windows(features, cfg.model.sequence_length)` : This transforms our raw feature matrix into the appropriate input-output pairs for training the LSTM/GRU models, where `x` contains sequences of past observations and `y` contains the corresponding next time step values. Hydra's `sequence_length` parameter controls how many past hours the model looks at to make its prediction.
- `MLFlowLogger(run_id=run.info.run_id)` : This is the critical line that bridges PyTorch Lightning's logging system with MLflow. By passing the active `run_id`, we ensure that all metrics logged by Lightning (like `train_loss`) are automatically captured in the same MLflow run context, giving us a unified view of our experiments.
- `pl.Trainer(logger=mlf_logger)` : Then, by passing our MLflow logger to the PyTorch Lightning Trainer, we ensure that all training metrics are seamlessly sent to MLflow without any additional code. This allows us to leverage Lightning's powerful logging capabilities while maintaining a clean separation of concerns.
- For Prophet, since it does not have a built-in training loop, we manually calculate the training loss (MSE) after fitting the model and log it to MLflow using `mlflow.log_metric()`. This ensures that even our non-deep learning models are tracked in the same experiment framework.

6. Orchestrating the Training Pipeline (Apache Airflow)

Finally, we need to automate this training pipeline. We will use Apache Airflow to schedule the training script to run on the first day of every month. This ensures that our models are regularly updated with the latest data without manual intervention.

To achieve a truly mature MLOps architecture, we will implement a hybrid approach: **Autonomous Execution** combined with **Granular UI Control**.

Our DAG will dynamically scan the Hydra configuration folder and generate a parallel task for every model it finds. However, we will also inject a UI parameter called `target_model`. By default, this is set to `"all"`, allowing the monthly schedule to train everything. But if a user manually triggers the DAG, they can select a specific model from the dropdown. The tasks that were not selected will utilize Airflow's `exit 99` bash command to gracefully mark themselves as "Skipped" without failing the pipeline.

Create a new DAG on `dags/model_training_dag.py` and add the following code:

```
# dags/model_training_dag.py
import os
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from datetime import datetime, timedelta
from airflow.sdk import Param
from airflow.sdk.definitions.param import ParamsDict

# 1. Dynamically scan the Hydra configuration directory
CONF_MODEL_DIR = "/opt/airflow/conf/model"
try:
    available_models = [
        f.replace('.yaml', '')
        for f in os.listdir(CONF_MODEL_DIR)
        if f.endswith('.yaml')
    ]
except FileNotFoundError:
    # Fallback safety
    available_models = ["lstm", "gru", "prophet"]

# 2. Add 'all' as the default option for our UI dropdown
dropdown_options = ["all"] + available_models

# Default arguments applied to all tasks in the DAG
default_args = {
    'owner': 'mlops_engineer',
    'depends_on_past': False,
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 5,
    'retry_delay': timedelta(seconds=5),
}
```

```

# 3. Define the DAG with the dynamic UI parameter
with DAG(
    'monthly_model_training',
    default_args=default_args,
    description='Trains all models autonomously, or a specific user-selected model.',
    schedule='@monthly',
    catchup=False,
    tags=['weather_capstone', 'training'],
    params=ParamsDict({
        "target_model": Param(
            default="all",
            enum=dropdown_options,
            description="Select 'all' to run every model, or pick a specific one to
train."
        )
    })
) as dag:
    # 4. Dynamically generate a training task for EVERY model found
    for model_name in available_models:
        # Bash logic: If UI is 'all' OR matches this specific task's model, run the training.
        # Otherwise, exit with 99. Airflow interprets exit 99 as a "Skipped" task state.
        run_logic = (
            f"if [ '{{{{ params.target_model }}}}' = 'all' ] || [ '{{{{ params.target_model }}}}' = '{model_name}' ]; then "
            f"cd /opt/airflow && python src/training/train_model.py model={model_name}; "
            "else "
            f"echo 'Skipping {model_name} training based on UI selection.'; "
            "exit 99; "
            "fi"
        )

        train_model_task = BashOperator(
            task_id=f'train_{model_name}_model',
            bash_command=run_logic
        )

```

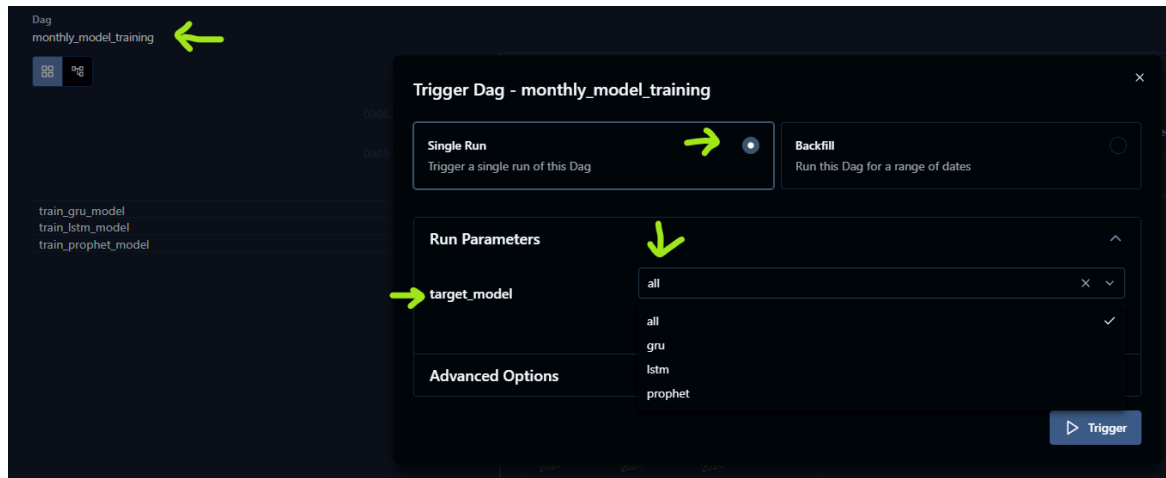
Code Explanation:

- `CONF_MODEL_DIR` : This variable points to the directory where our Hydra model configuration files are stored. The DAG will scan this directory at runtime to determine which models are available for training.
- `available_models` : This list is populated by reading the contents of the `CONF_MODEL_DIR` . It extracts the model names by removing the `.yaml` extension. If the directory is not found (e.g., during development or testing), it falls back to a hardcoded list of models.
- `dropdown_options` : This list includes all the dynamically discovered models plus an "all" option. This will be used to populate the dropdown in the Airflow UI, allowing users to select either a specific model or all models for training.
- `params=ParamsDict(...)` : This defines a UI parameter called `target_model` for the DAG. It defaults to "all" and is constrained to the options defined in `dropdown_options` . This parameter will be visible in the Airflow UI when manually triggering the DAG, allowing users to control which model(s) to train.
- `run_logic` : This is the core of our hybrid execution strategy. It checks the value of `target_model` :
 - If `target_model` is "all" or matches the current `model_name` , it executes the training command for that model.
 - If `target_model` does not match, it prints a message and exits with code 99. Airflow treats an exit code of 99 as a "Skipped" state, which allows us to gracefully skip tasks without marking the entire DAG as failed. This means that if a user selects "gru" from the dropdown, only the `train_gru_model` task will run, while the `train_lstm_model` and `train_prophet_model` tasks will be marked as "Skipped" in the Airflow UI, providing clear visibility into what was executed and what was intentionally bypassed.

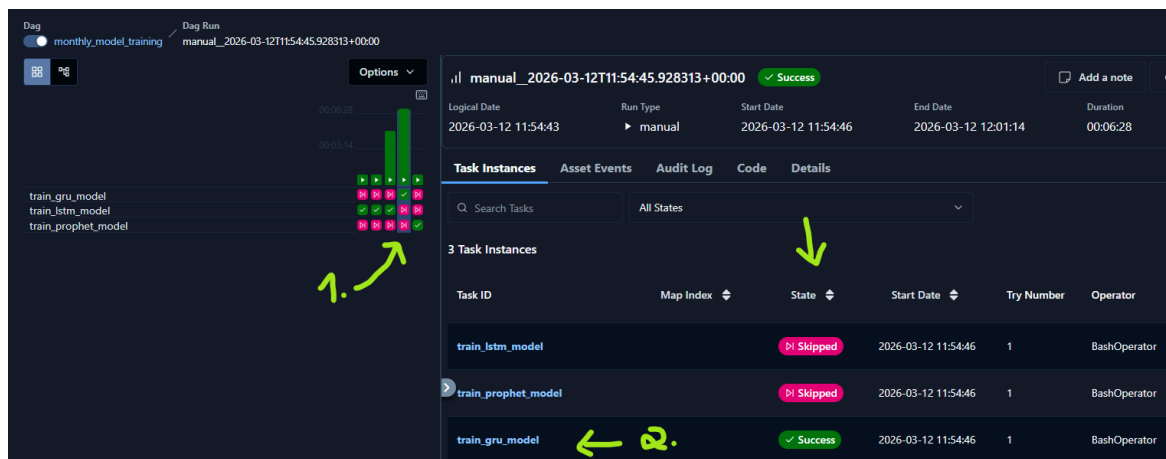
This architecture allows us to maintain a fully automated monthly training schedule while also giving users the flexibility to run specific models on demand without any code changes. The UI-driven control combined with the autonomous execution ensures that we can meet both operational efficiency and user interactivity requirements in our MLOps workflow.

7. Testing the Hybrid DAG via the Airflow UI

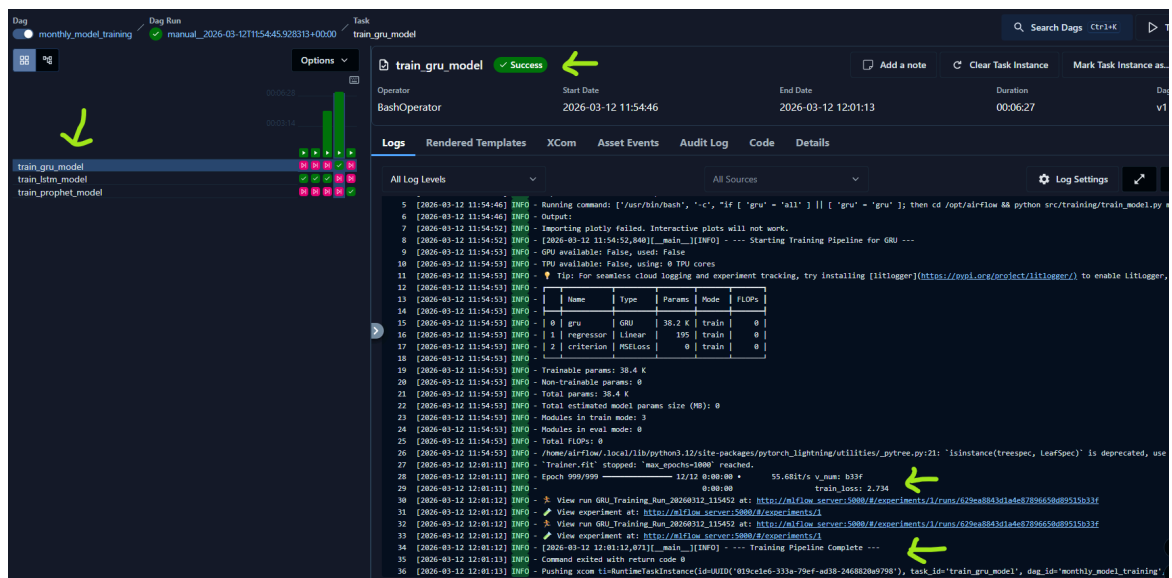
1. **Access the UI:** Open your browser and navigate to `http://localhost:8080`.
2. **Trigger with Config:** Locate the `monthly_model_training` DAG. Click the **Trigger** button (the play icon) and select **Single Run**.
3. **Test the Dropdown:** You will see the `target_model` parameter defaulting to `"all"`. Click it, and you will see your dynamically loaded list (`lstm`, `gru`, `prophet`).



4. **Monitor the "Skip" Logic:** Select `gru` and trigger the run. Click into the DAG to view the grid/graph. You will see something like: the `train_gru_model` task will turn dark green (Success), while `train_lstm_model` and `train_prophet_model` will turn pink (Skipped)!
- This confirms that our UI parameter is correctly controlling the execution flow without causing any failures!



You can also check the logs of each task to see the print statements confirming the training process or the skipping logic. The image below shows the logs for the `train_gru_model` task, confirming that it ran successfully:



5. **Review MLflow Tracking:** After the tasks complete, navigate to your MLflow UI (<http://localhost:5000>) to review the tracked experiments for each model. You should see separate runs for LSTM, GRU, and Prophet, each with their respective parameters and training loss metrics logged.



8. Next Steps

Now that we have a robust training pipeline with dynamic model selection and comprehensive experiment tracking, the next step in Lab 6 will be to evaluate the trained models on a test set, save the best-performing model artifacts, and implement a forecasting pipeline to generate future predictions. We will also explore how to automate the entire workflow end-to-end using Airflow, ensuring that our models are not only trained but also deployed and monitored in production seamlessly.